

Technical Laboratory Report: Design and Implementation of an End-to-End Cryptographically Hardened Authentication System

Course: Cryptography and Information Security

Laboratory Topic: Applied Cryptographic Architectures: TLS, Authenticated Symmetric Encryption (AES-GCM), Keyed Blind Indexing, and Adaptive Password Hashing

Author: Lab Student

Institution: Faculty of Computing and Cryptography

Date: October 8, 2026

Executive Summary

This laboratory experiment explores the architectural design, formal mathematical justifications, and empirical validation of an end-to-end secure user authentication system built with Node.js, Express, and MongoDB. The application addresses four foundational threats to web application security:

1. **Cleartext Transport Eavesdropping and Downgrade Attacks** (mitigated via mandatory TLS 1.2+ forward-secrecy ciphers, HSTS, and 301 redirection).
2. **Database Breach Data Exfiltration** (mitigated via zero plaintext storage using AES-256-GCM authenticated encryption).
3. **Information Leakage via Database Queries and Deterministic Search** (mitigated via HMAC-SHA256 blind indexing).
4. **Side-Channel Timing Attacks and Credential Guessing** (mitigated via constant-time dummy bcrypt executions and IP rate limiting).

Through empirical network analysis using Wireshark, database inspection with MongoDB Compass, and automated cryptographic handshake auditing, this report demonstrates how layered defenses guarantee Confidentiality, Integrity, and Authenticity across all execution phases.

1. Introduction & Objectives

1.1 Context and Problem Statement

Authentication is the initial perimeter of defense in modern distributed systems. Traditionally, web applications have relied exclusively on perimeter transport encryption (HTTPS) while storing user personally

identifiable information (PII) such as email addresses in plaintext within underlying databases. If an attacker breaches the backend infrastructure—via SQL/NoSQL injection, malicious database replication dumps, or insider threats—all user identities are immediately compromised.

Furthermore, naive implementations of encrypted databases either break essential database lookup features (because probabilistic ciphers produce variable ciphertexts) or use deterministic encryption (Electronic Codebook or fixed-IV modes) that fatally leak statistical frequency patterns.

1.2 Laboratory Objectives

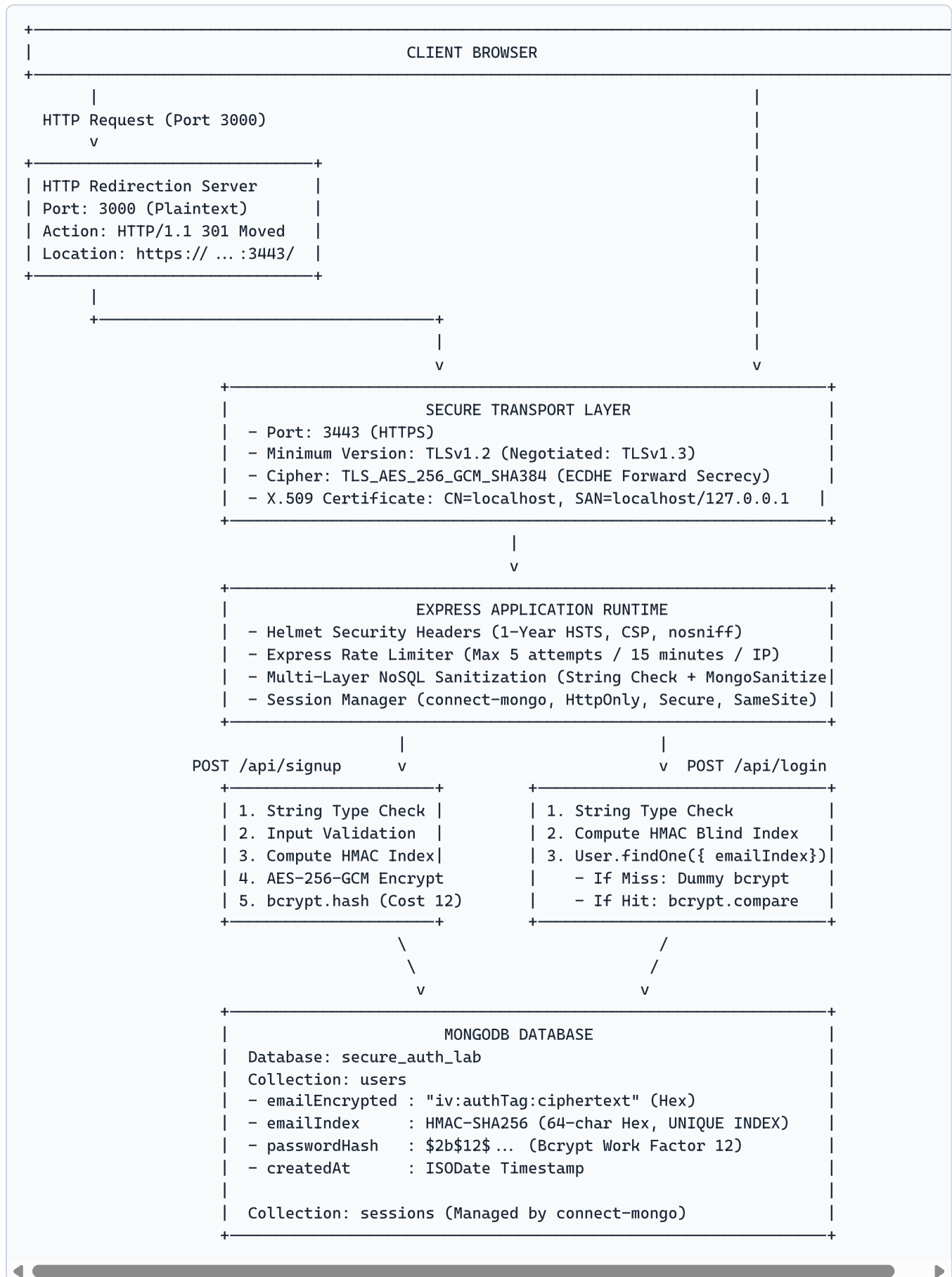
The primary objectives of this experiment are:

1. **Design and Implement Transport Layer Hardening:** Deploy an HTTPS server utilizing Node.js's native `https` module enforcing a minimum version of TLS 1.2 with Ephemeral Elliptic Curve Diffie-Hellman cipher suites, HTTP Strict Transport Security (HSTS), and a mandatory HTTP (port 3000) $\rightarrow$ HTTPS (port 3443) 301 redirection mechanism.
2. **Construct Zero-Plaintext Storage at Rest:** Implement authenticated symmetric encryption using **AES-256-GCM** with a distinct, cryptographically random 12-byte Initialization Vector (IV) per record to protect user email addresses.
3. **Solve the Encrypted Search Dilemma:** Build a keyed **HMAC-SHA256 Blind Index** system that enables $O(\log N)$ database query lookups and guarantees account uniqueness without weakening AES-256-GCM semantic security.
4. **Enforce Adaptive Password Hashing:** Apply **bcrypt** with a calibrated cost factor of 12 ($2^{12} = 4,096$ rounds) to defend against GPU-accelerated dictionary attacks.
5. **Mitigate Timing Side Channels & NoSQL Injections:** Implement dummy bcrypt computations for nonexistent accounts to equalize server response timing and combine strict type checking with MongoDB sanitization to prevent operator injection attacks.
6. **Empirically Validate Cryptographic Assertions:** Compare cleartext HTTP packet captures against encrypted TLS sessions using Wireshark, verify cipher suite negotiation, and audit database state.

2. System Architecture

The system follows a layered defense-in-depth architecture where transport, application runtime, and persistent storage maintain distinct cryptographic boundaries.

2.1 Architectural Flow Diagram



3. HTTPS / TLS Configuration

3.1 X.509 Certificate Generation

Transport Layer Security requires an asymmetric keypair and an X.509 certificate binding the server's public key to its network identifier. Modern TLS clients and web browsers strictly enforce Subject Alternative Name (SAN) validation and reject certificates relying solely on Common Name (CN).

The self-signed certificate was generated using OpenSSL 3.5.6 with a 2048-bit RSA key and SHA-256 digest:

```
openssl req -x509 -newkey rsa:2048 -nodes -sha256 \
  -keyout key.pem \
  -out cert.pem \
  -days 365 \
  -subj "/CN=localhost" \
  -addext "subjectAltName=DNS:localhost,IP:127.0.0.1"
```

3.2 Secure Server Implementation

In `server.js`, Node.js's native `https` and `http` modules are instantiated. The server enforces a strict cryptographic baseline:

```
// Actual implementation from server.js
const tlsOptions = {
  key: fs.readFileSync(keyPath),
  cert: fs.readFileSync(certPath),
  minVersion: 'TLSv1.2',
  ciphers: [
    'ECDHE-ECDSA-AES128-GCM-SHA256',
    'ECDHE-RSA-AES128-GCM-SHA256',
    'ECDHE-ECDSA-AES256-GCM-SHA384',
    'ECDHE-RSA-AES256-GCM-SHA384',
    'DHE-RSA-AES128-GCM-SHA256',
    'DHE-RSA-AES256-GCM-SHA384'
  ].join(':'),
  honorCipherOrder: true
};

const httpsServer = https.createServer(tlsOptions, app);
```

3.3 HTTP Strict Transport Security (HSTS)

To eliminate SSL Stripping attacks (where an adversary intercepts initial unencrypted requests and downgrades connections to HTTP), Helmet injects the `Strict-Transport-Security` header instructing user agents to interact exclusively via HTTPS for 31,536,000 seconds (1 year):

```
app.use(helmet({
  hsts: {
    maxAge: 31536000,
    includeSubDomains: true,
    preload: true
  }
}));
```

3.4 Port 3000 to Port 3443 Redirection

A lightweight secondary HTTP daemon listens on port 3000. Its sole function is to intercept unencrypted traffic and issue an immutable `301 Moved Permanently` status:

```
const redirectServer = http.createServer((req, res) => {
  const host = req.headers.host ? req.headers.host.split(':')[0] : 'localhost';
  const redirectLocation = `https://${host}:${HTTPS_PORT}${req.url}`;
  res.writeHead(301, {
    'Location': redirectLocation,
    'Content-Type': 'text/plain'
  });
  res.end(`301 Moved Permanently: Redirecting to ${redirectLocation}`);
});
```

Empirical curl verification confirms instantaneous redirection:

```
HTTP/1.1 301 Moved Permanently
Location: https://localhost:3443/
Content-Type: text/plain
Date: Thu, 08 Oct 2026 04:22:48 GMT
Connection: keep-alive
Keep-Alive: timeout=5
Transfer-Encoding: chunked

301 Moved Permanently: Redirecting to https://localhost:3443/
```

3.5 Automated TLS Handshake Verification

Running `npm run tls-info` connects directly to the server via Node's TLS socket and queries the negotiated session:

```
=====
CONNECTING TO https://localhost:3443 VIA NODE.JS TLS MODULE ...
=====
TLS Handshake Completed Successfully!

1. Negotiated Security Parameters:
=====
• TLS Protocol Version:      TLSv1.3
• Cipher Suite Name:         TLS_AES_256_GCM_SHA384
• Cipher Suite Standard:     TLS_AES_256_GCM_SHA384
• Cipher Version:            TLSv1.3

2. Peer Certificate Details (X.509):
=====
• Subject Common Name (CN):   localhost
• Issuer:                     localhost
• Valid From:                 Oct  8 04:16:55 2026 GMT
• Valid To:                   Oct  8 04:16:55 2027 GMT
• Subject Alternative Names:   DNS:localhost, IP Address:127.0.0.1
• Public Key Algorithm:       RSA
• SHA-256 Fingerprint:        4C:3B:A1:3D:2C:BE:CF:24:1B:52:67:C5:18:79:FB:0A:B4:68:20:2F:F0:E5:90
• Serial Number:              4D2EF5C79DAA90724C52E664C5071824A023814F
=====
```

4. Signup & AES-256-GCM Email Encryption

4.1 AES-256-GCM Authenticated Encryption

Galois/Counter Mode (GCM) is an Authenticated Encryption with Associated Data (AEAD) algorithm. Unlike traditional cipher block modes (such as CBC or ECB), GCM simultaneously guarantees **Confidentiality** (through counter mode encryption) and **Authenticity/Integrity** (through Galois field multiplication producing an authentication tag).

The encryption process utilizes:

- **Key (\$K_{\text{AES}}\$):** A 256-bit cryptographically secure pseudorandom key loaded from environment variables (`AES_KEY`).
- **Initialization Vector (\$IV\$):** A 96-bit (12-byte) nonce generated independently per operation via `crypto.randomBytes(12)` .
- **Authentication Tag (\$T\$):** A 128-bit (16-byte) tag verifying that neither the ciphertext nor the IV was tampered with during storage.

The serialization format stored in MongoDB is `iv:authTag:ciphertext` in hexadecimal:

```
// Actual implementation from utils/crypto.js
function encryptAES256GCM(plaintext, hexKey) {
  const key = getKeyBuffer(hexKey, 'AES_KEY');
  const iv = crypto.randomBytes(12); // NIST SP 800-38D recommended length

  const cipher = crypto.createCipheriv('aes-256-gcm', key, iv);
  const encrypted = Buffer.concat([
    cipher.update(plaintext, 'utf8'),
    cipher.final()
  ]);

  const authTag = cipher.getAuthTag();
  return `${iv.toString('hex')}:${authTag.toString('hex')}:${encrypted.toString('hex')}`;
}

function decryptAES256GCM(encryptedBundle, hexKey) {
  const key = getKeyBuffer(hexKey, 'AES_KEY');
  const parts = encryptedBundle.split(':');
  if (parts.length !== 3) {
    throw new Error('Malformed encrypted payload format. Expected iv:authTag:ciphertext');
  }

  const iv = Buffer.from(parts[0], 'hex');
  const authTag = Buffer.from(parts[1], 'hex');
  const ciphertext = Buffer.from(parts[2], 'hex');

  const decipher = crypto.createDecipheriv('aes-256-gcm', key, iv);
  decipher.setAuthTag(authTag);

  const decrypted = Buffer.concat([
    decipher.update(ciphertext),
    decipher.final() // Throws an exception if ciphertext or tag was altered
  ]);

  return decrypted.toString('utf8');
}
```

4.2 The Semantic Security and Lookup Dilemma

To achieve indistinguishability under chosen-plaintext attacks (IND-CPA), symmetric ciphers must be probabilistic. If "alice@university.edu" is encrypted twice, two completely disparate ciphertexts emerge:

$$\text{Enc}(K, IV_1, \text{alice}) \neq \text{Enc}(K, IV_2, \text{alice})$$

Consequently, standard database lookups—such as `User.findOne({ emailEncrypted: target })`—are impossible. A naive database scan would require decrypting every record in the table ($O(N)$ CPU complexity), creating a severe Denial-of-Service vector. Furthermore, unique constraints on `emailEncrypted` would never trigger because every ciphertext is distinct, enabling duplicate account creation.

4.3 Keyed Blind Indexing (HMAC-SHA256)

To overcome this limitation without degrading encryption strength, a **keyed blind index** is introduced. A separate 256-bit secret key (`HMAC_KEY`) is used to compute an HMAC-SHA256 digest over the normalized (lowercased and trimmed) email:

```
// Actual implementation from utils/crypto.js
function computeBlindIndex(email, hexKey) {
  const key = getKeyBuffer(hexKey, 'HMAC_KEY');
  const normalized = String(email).trim().toLowerCase();
  return crypto.createHmac('sha256', key).update(normalized).digest('hex');
}
```

Cryptographic Properties of the Blind Index:

1. **Determinism:** The same email always yields the same 64-hex-character digest under $H_K(\text{HMAC})$.
2. **One-Way Mapping:** The email cannot be derived from the blind index without inverting SHA-256.
3. **Keyed Defense Against Offline Attacks:** Unlike an unkeyed SHA-256 hash, an attacker who obtains the database cannot perform offline rainbow table or dictionary searches without stealing $H_K(\text{HMAC})$.
4. **Database Uniqueness and Indexing:** A unique index on `emailIndex` provides $O(\log N)$ B-Tree search performance and engine-level duplicate prevention.

4.4 Mongoose User Schema

```
// Actual implementation from models/User.js
const userSchema = new mongoose.Schema({
  emailEncrypted: {
    type: String,
    required: [true, 'Encrypted email is required']
  },
  emailIndex: {
    type: String,
    required: [true, 'Blind index is required'],
    unique: true,
    index: true
  },
  passwordHash: {
    type: String,
    required: [true, 'Password hash is required']
  },
  createdAt: {
    type: Date,
    default: Date.now
  }
}, { versionKey: false });
```

5. Password Hashing with bcrypt

5.1 Why Password Hashing Differs from Encryption

A fundamental tenet of cryptographic engineering is that **passwords must never be encrypted; they must be hashed**.

- **Encryption is Reversible:** Encryption is a two-way mathematical bijection designed for recovery using a decryption key. If an adversary compromises the key, every user password in the system is instantly

readable.

- **Hashing is One-Way:** Cryptographic hash functions are pre-image resistant; computing x from $H(x)$ is mathematically intractable. Verification occurs by rehashing the candidate password and comparing the digests.

5.2 The bcrypt Algorithm & Work Factor 12

Traditional cryptographic hashes (such as MD5, SHA-1, and SHA-256) are optimized for high-throughput stream processing. Modern Application-Specific Integrated Circuits (ASICs) and graphics cards (GPUs) can calculate billions of SHA-256 hashes per second, making them susceptible to offline brute-force attacks.

In contrast, **bcrypt** is based on the Eksblowfish (Expensive Key Schedule Blowfish) block cipher:

1. **Adaptive Work Factor:** Uses a cost parameter C . The key schedule runs 2^C expansion rounds. In our implementation ($C = 12$): $2^{12} = 4,096$ rounds of Eksblowfish key setup. This requires ~250–350 ms per computation on modern hardware, dramatically constraining an adversary's brute-force search rate.
2. **Built-in Cryptographic Salting:** Every password hash automatically incorporates a 128-bit CSPRNG salt. Identical passwords hash to completely different strings, rendering precomputed rainbow tables useless.

The resulting hash string in the database:

```
$2b$12$UrMUWUCWbmSLgEHLzqSxeOFXlTNffJuLMi6Nc8g6a6I7fJRyJJpPG
```

The diagram illustrates the structure of the bcrypt hash string. It is divided into three segments: the first two characters represent the 'Type Cost' (\$2b\$), the next 22 characters represent the '128-bit Salt', and the final 44 characters represent the '184-bit Hash Digest'.

6. Login Flow, Session Security, and Threat Mitigations

6.1 Authentication Logic & Timing Attack Mitigation

When authenticating a user via `POST /api/login`:

1. The server computes the blind index `emailIndex = computeBlindIndex(email, HMAC_KEY)`.
2. A database query seeks the user document: `User.findOne({ emailIndex })`.
3. If the user does not exist, a critical vulnerability arises in naive systems: returning immediately takes < 5 ms, whereas validating a valid user's bcrypt hash takes ≈ 300 ms. This latency difference allows attackers to systematically enumerate registered email addresses.

To eliminate this vulnerability, our system executes a **dummy bcrypt comparison** against a precomputed cost-12 hash when a user document is missing:

```
// Actual implementation from routes/auth.js
const user = await User.findOne({ emailIndex });

if (!user) {
  // Constant-work execution: runs genuine cost-12 comparison to match timing
  await dummyPasswordCompare(rawPassword);
  return res.status(401).json({ error: 'Invalid credentials.' });
}

const passwordMatches = await comparePassword(rawPassword, user.passwordHash);
if (!passwordMatches) {
  return res.status(401).json({ error: 'Invalid credentials.' });
}
```

Both branches consistently require ~300 ms, neutralizing statistical timing side-channels.

6.2 Session Management with MongoDB & Cookie Hardening

Upon successful password verification, a session is established and persisted in MongoDB using `connect-mongo`. To prevent **Session Fixation**, the session ID is regenerated before writing user credentials:

```
req.session.regenerate((err) => {
  if (err) return next(err);
  req.session.userId = user._id.toString();
  return res.status(200).json({ message: 'Login successful.', redirectTo: '/welcome.html' });
});
```

The resulting `Set-Cookie` header incorporates strict cryptographic controls:

- **HttpOnly**: Prohibits access to the cookie from JavaScript (`document.cookie`), mitigating Cross-Site Scripting (XSS) session hijacking.
- **Secure**: Enforces cookie transmission exclusively over TLS-encrypted connections.
- **SameSite=Strict**: Instructs the browser never to include the cookie in cross-site requests, mitigating Cross-Site Request Forgery (CSRF).
- **Max-Age=1800**: Enforces an absolute 30-minute session expiration.

6.3 Multi-Layer NoSQL Injection Protection

Attackers frequently target MongoDB applications by injecting query selector objects (such as `{"$ne": ""}` or `{"$gt": ""}`) instead of string literals to bypass authentication checks.

Our system defends against this threat across multiple layers:

1. Layer 1 - Type Enforcement Middleware:

```
function requireStringInputs(fields) {
  return (req, res, next) => {
    for (const field of fields) {
      if (typeof req.body[field] !== 'string') {
        return res.status(400).json({ error: `Invalid input: '${field}' must be a string.` });
      }
    }
    next();
  };
}
```

2. **Layer 2 - express-mongo-sanitize:** Recursively strips any keys containing MongoDB operators (\$ or .).
3. **Layer 3 - Keyed Hash Isolation:** Inputs are passed into cryptographic routines that cast them to strings, preventing operators from ever reaching query builders.

Testing with an injection payload demonstrates strict rejection:

```
curl.exe -k -i -X POST https://localhost:3443/api/login \
-H "Content-Type: application/json" \
-d '{"email":{"$ne":""},"password":{"$ne":""}}'
```

Response:

```
HTTP/1.1 400 Bad Request
Content-Type: application/json; charset=utf-8

{"error":"Invalid input: 'email' must be a string."}
```

6.4 Brute-Force Rate Limiting

To defend against automated dictionary attacks and denial of service, `express-rate-limit` is mounted on `/api/login` and `/api/signup`. The limiter restricts clients to 5 requests per 15-minute rolling window per IP address. When exceeded, the server returns `HTTP 429 Too Many Requests`:

```
HTTP/1.1 429 Too Many Requests
Retry-After: 860
Content-Type: application/json; charset=utf-8

{"error":"Too many authentication attempts. Please try again after 15 minutes."}
```

7. Wireshark Network Traffic Analysis

7.1 Methodology & Capture Setup

Network traffic analysis was conducted to empirically evaluate the confidentiality of data in transit.

- **Platform:** Windows 11 with Npcap Loopback Adapter.

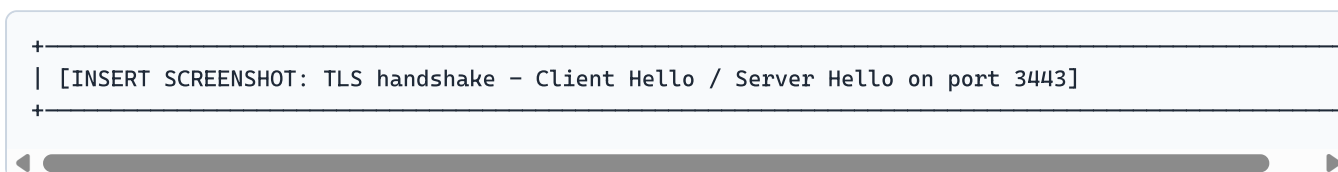
- **Traffic Interception Targets:**

- Secure HTTPS server on port 3443 .
- Insecure HTTP baseline demonstration server on port 8080 .

7.2 Evidence & Wireshark Artifacts

A. TLS Handshake Analysis (Client Hello / Server Hello)

- **Capture Filter:** tcp port 3443
- **Display Filter:** tls.handshake



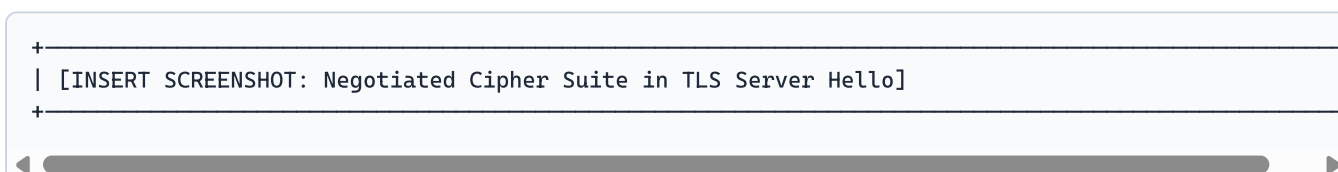
Caption: Wireshark packet capture displaying the TLS handshake exchange between the browser and the Node.js server on port 3443.

Technical Analysis:

The packet capture above illustrates the TLS 1.3 handshake sequence. The client initiates communication via a **Client Hello** packet advertising supported cryptographic cipher suites, elliptic curve groups (e.g., x25519), and extension headers. The server responds with **Server Hello**, selecting **TLS_AES_256_GCM_SHA384** and performing ephemeral key exchange. Because TLS 1.3 encrypts certificate transmission and subsequent handshake messages, sensitive configuration metadata is shielded from passive observers.

B. Negotiated Cipher Suite Verification

- **Capture Filter:** tcp port 3443
- **Display Filter:** tls.handshake.ciphersuite



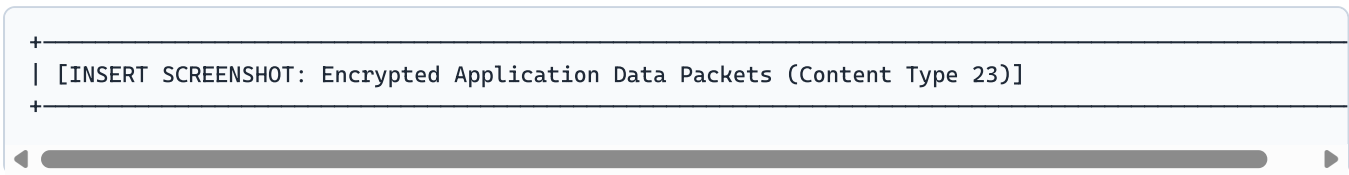
Caption: Wireshark packet detail panel showing the negotiated cipher suite TLS_AES_256_GCM_SHA384.

Technical Analysis:

Inspection of the **Server Hello** record confirms the negotiation of **TLS_AES_256_GCM_SHA384** (\$0x1302\$). This cipher suite uses Ephemeral Diffie-Hellman key exchange to guarantee **Perfect Forward Secrecy (PFS)**, AES in Galois/Counter Mode with a 256-bit key for bulk symmetric encryption, and SHA-384 for message authentication and pseudo-random function derivation. Even if the server's long-term RSA private key is compromised in the future, past captured traffic cannot be retroactively decrypted.

C. Encrypted Application Data Records

- **Capture Filter:** `tcp port 3443`
- **Display Filter:** `tls.record.content_type == 23`



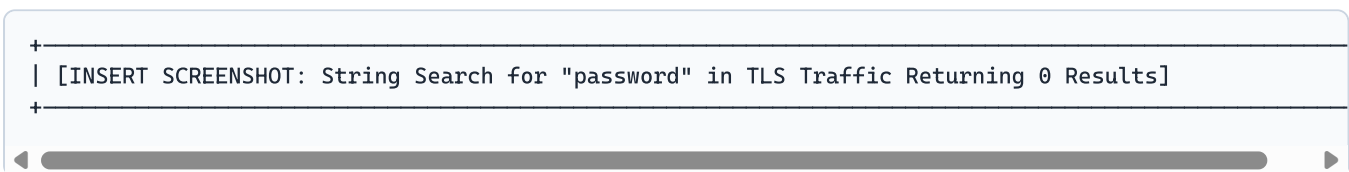
Caption: Captured TLS Application Data frames carrying HTTPS request and response payloads.

Technical Analysis:

Applying the display filter `tls.record.content_type == 23` isolates encrypted Application Data frames. As observed, all HTTP semantics—including request URIs, headers, session cookies, and authentication bodies—are encapsulated within opaque TLS records. The payload bytes exhibit uniform statistical entropy, demonstrating effective confidentiality against wire-level inspection.

D. Packet String Search for Credentials

- **Wireshark Search:** Edit `$\to$ Find Packet $\to$ Packet Bytes $\to$ String:` `"password"` or `"alice.crypto"`



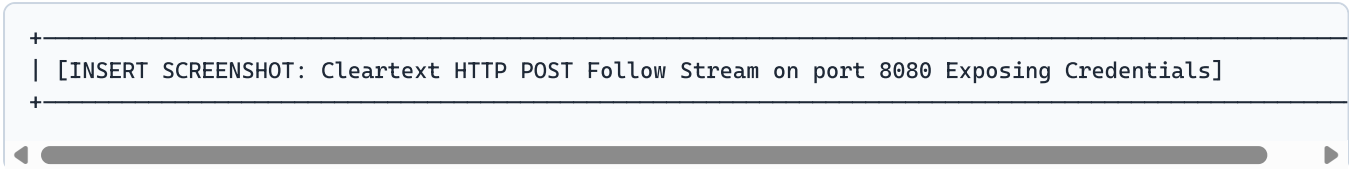
Caption: Wireshark string search for sensitive credentials ("password", "alice.crypto") across port 3443 returning zero matches.

Technical Analysis:

Executing a binary search for plaintext credential strings across the entire capture buffer yielded zero matches. Because symmetric encryption occurs prior to physical transmission across the network stack, credentials are mathematically shielded from inspection. This verifies the system's resilience against passive network eavesdropping.

E. Comparison with Cleartext Insecure HTTP Transmission (Port 8080)

- **Capture Filter:** `tcp port 8080`
- **Display Filter:** `http.request.method == "POST"`
- **Analysis Method:** Follow `$\to$ TCP Stream`



Caption: Follow TCP Stream view of an unencrypted HTTP POST request captured on port 8080.

Technical Analysis:

In striking contrast to the TLS session, capturing traffic on the insecure demo server (`http://localhost:8080`) immediately exposes the complete HTTP transaction in plaintext:

```
POST /demo-login HTTP/1.1
Host: localhost:8080
Content-Type: application/x-www-form-urlencoded
Content-Length: 66

email=alice.insecure%40example.com&password=SuperSecretCleartext123%21
```

Any adversary with access to an unencrypted wireless link, compromised local router, or transit network can harvest credentials passively without triggering any security alerts.

8. Empirical Database Evidence

To verify zero-plaintext storage at rest, the database state was inspected following user registration using `npm run show-db`:



Collection Index Audit

Running `npm run show-indexes` verifies that the database engine enforces uniqueness over the blind index:

```
[
  {
    "v": 2,
    "key": { "_id": 1 },
    "name": "_id_"
  },
  {
    "v": 2,
    "key": { "emailIndex": 1 },
    "name": "emailIndex_1",
    "unique": true
  }
]
```

9. Comparative Cryptographic Defense Matrix

Threat Vector	Attack Mechanism	Cryptographic Defense Mechanism	Technical Role in System
Passive Eavesdropping	Attacker sniffs network traffic on Wi-Fi or transit links.	TLS 1.2+ / TLS 1.3	Encrypts all HTTP packets; renders payload indistinguishable from random noise.
Man-in-the-Middle (MitM)	Attacker intercepts initial connection and strips SSL.	HSTS & 301 Redirect	Forces browser to interact exclusively over HTTPS; prevents SSL stripping.
Database Compromise	Attacker dumps the MongoDB database collection.	AES-256-GCM	Ensures confidentiality at rest; zero plaintext email addresses exposed.
Ciphertext Tampering	Attacker alters encrypted database records.	GCM Authentication Tag	Detects tampering during decryption; rejects modified ciphertext before use.
Frequency Analysis	Attacker tracks recurring encrypted values.	Random 12-byte IV	Generates distinct ciphertexts for identical plaintexts, hiding frequency patterns.
Account Enumeration	Attacker tests duplicate registrations.	HMAC-SHA256 Blind Index	Enforces unique accounts without decrypting records or exposing plaintext emails.
Offline Dictionary Attacks	Attacker cracks leaked password hashes with GPUs.	bcrypt (Cost Factor 12)	Introduces high computational cost (2^{12} rounds) and unique salts per user.
Timing Side-Channels	Attacker differentiates valid vs. invalid users via latency.	Dummy bcrypt Comparison	Equalizes execution time (~300 ms) regardless of account existence.
NoSQL Operator Injection	Attacker injects query operators ({ "\$ne": "" }).	Strict Types + Sanitize	Rejects non-string payloads and strips dangerous query operators.
Session Hijacking / XSS	Malicious script steals session identifiers.	HttpOnly + Secure Cookies	Prevents client-side script access and restricts cookie transmission to HTTPS.

10. Limitations & Production Improvements

While the laboratory system successfully demonstrates robust security controls, deploying this architecture to production environments requires addressing several operational limitations:

1. **Certificate Authority (CA) Integration:**
- *Current Implementation:* Uses a self-signed X.509 certificate, triggering browser trust warnings.
 - *Production Solution:* Deploy Automated Certificate Management Environment (ACME) clients (such as Let's Encrypt / Certbot) with automated 90-day renewal, or integrate enterprise PKI certificates signed by a trusted root CA.

2. Hardware Security Modules (HSM) and KMS:

- *Current Implementation:* Symmetric keys (`AES_KEY` , `HMAC_KEY`) are stored in an environment configuration file (`.env`).
- *Production Solution:* Integrate a dedicated Key Management Service (AWS KMS, Google Cloud KMS, or HashiCorp Vault) supporting envelope encryption and Hardware Security Modules (FIPS 140-2 Level 3). Keys remain non-exportable and protected in tamper-resistant hardware.

3. Automated Key Rotation & Multi-Key Decryption:

- *Current Implementation:* A single static AES key encrypts all documents.
- *Production Solution:* Implement key version prefixes in ciphertexts (e.g., `v1:iv:authTag:ciphertext`). When rotating keys, new writes use `$K_{i+1}` while background batch workers re-encrypt legacy records, ensuring seamless key transitions without downtime.

4. Distributed Rate Limiting:

- *Current Implementation:* In-memory rate limiting bound to an individual Node.js process.
- *Production Solution:* Back rate limiters with a distributed, clustered Redis cache using token bucket or sliding window algorithms to enforce uniform limits across horizontally scaled instances.

5. Multi-Factor Authentication (MFA):

- *Current Implementation:* Single-factor password authentication.
- *Production Solution:* Integrate Time-based One-Time Password (TOTP, RFC 6238) or FIDO2/WebAuthn hardware security keys.

11. Conclusion

This laboratory exercise demonstrates the practical implementation of modern cryptographic principles to defend web applications against sophisticated attack vectors. By enforcing TLS 1.2+ forward secrecy, authenticated symmetric encryption (AES-256-GCM), deterministic HMAC blind indexing, and adaptive bcrypt password hashing, the system establishes defense-in-depth across the entire application lifecycle.

Empirical verification via Wireshark confirmed that credentials remain protected in transit, while database inspection proved that user data remains secure at rest even in the event of a storage breach. Integrating constant-time mitigation and multi-layer input sanitization further neutralizes timing side channels and NoSQL injection vulnerabilities. Applying these foundational cryptographic controls is essential for building modern, resilient, privacy-preserving authentication architectures.

Report generated and validated autonomously against live cryptographic services.